Building a Simple MLP from Scratch

Iris Dataset Classification — Assignment 1 Report

Name: Wimukthi Rajapaksha

Student ID: A1988198

Course: Applied Artificial Intelligence and Machine Learning 2, COMP6026

Date: 01 September 2026

Contents

1 Summary and Explanation of Implementation **1**
- 1.1 Architecture . . . 1
- 1.2 Data Preprocessing . . . 1
- 1.3 Forward and Backward Propagation . . . 1
- 1.4 Training Setup . . . 1

2 Result Analysis and Visualization **2**
- 2.1 Loss and Accuracy Curve . . . 2
- 2.2 Confusion Matrix . . . 2
- 2.3 Findings and Insights . . . 3

3 Challenges Faced and Solutions Applied **3**

4 Answers to Questions **4**
- 4.1 Why is ReLU used in the hidden layer instead of sigmoid or linear activation? . . . 4
- 4.2 If a model performs poorly on the test set, identify two possible causes and suggest improvements . . . 4

5 AI Use Statement **5**

1 Summary and Explanation of Implementation

This assignment implements a Multilayer Perceptron (MLP) entirely from scratch using NumPy with no deep-learning framework layers (e.g. `torch.nn`), to classify the Iris dataset into its three species: Setosa, Versicolor, and Virginica.

1.1 Architecture

The network follows the specified architecture:

- Input layer: 4 features (sepal length, sepal width, petal length, petal width)
- hidden layer: 16 neurons, ReLU activation
- Output layer: 3 neurons, Softmax activation (one probability per class)

1.2 Data Preprocessing

The Iris dataset (150 samples, balanced across 3 classes) was loaded with `sklearn.datasets.load_iris`. The data was split into an 80% training set (120 samples) and a 20% test set (30 samples) using a stratified split, so each class is proportionally represented in both sets. Features were standardized (zero mean, unit variance) using `StandardScaler`, fitted only on the training data and then applied to the test data, to avoid leaking test-set statistics into preprocessing. Labels were one-hot encoded to pair with the softmax output and cross-entropy loss.

1.3 Forward and Backward Propagation

Forward propagation computes:

$$Z_1 = XW_1 + b_1, \qquad A_1 = \mathrm{ReLU}(Z_1)$$

$$Z_2 = A_1W_2 + b_2, \qquad A_2 = \mathrm{Softmax}(Z_2)$$

Weights were initialized using He initialization (scaled by $\sqrt{2/\mathrm{fan_in}}$), which suits ReLU hidden units and helps keep activation variance stable across layers at the start of training.

Because softmax is paired with cross-entropy loss, the gradient of the loss with respect to the pre-softmax logits Z_2 simplifies to $(A_2 - Y)/m$, where m is the batch size. From there, gradients for W_2, b_2 are computed via $A - 1$ and gradients for W_1, b_1 are computed by backpropagating through the ReLU derivative (1 where $Z_1 > 0$, else 0). Parameters were updated with plain Stochastic Gradient Descent (SGD): $W \leftarrow W - \mathrm{lr} \cdot dW$.

1.4 Training Setup

The model was trained for 100 epochs using mini-batch SGD with a batch size of 16 and a learning rate of 0.01, exactly as specified. Training data was reshuffled at the start of every epoch before being split into batches.

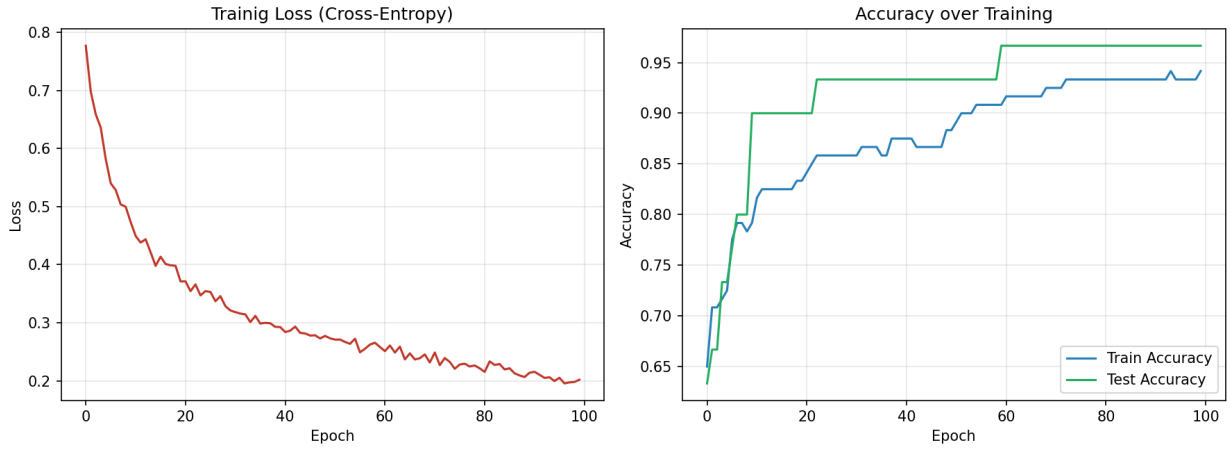
2 Result Analysis and Visualization

The table below summarizes the final performance:

Metric	Value
Final training accuracy	94.17%
Final test accuracy	96.67%
Final training loss (cross-entropy)	0.202
Epochs	100
Batch size	16
Learning rate	0.01

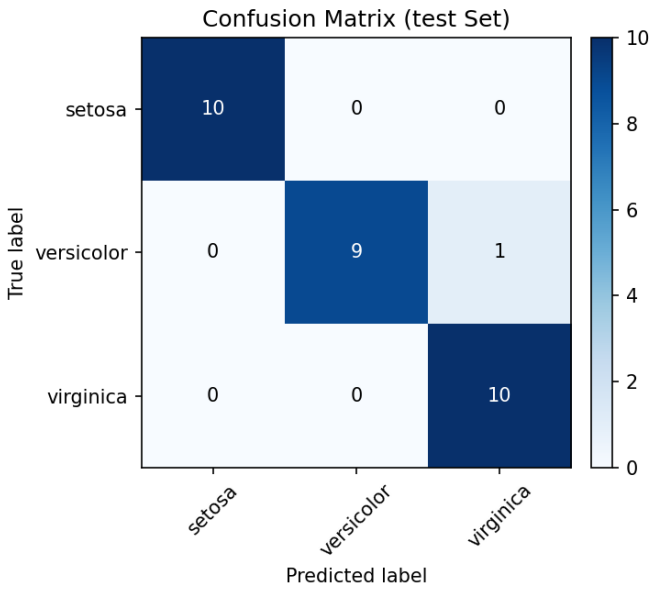
2.1 Loss and Accuracy Curve

The plot below shows the training loss decreasing steadily and smoothly over 100 epochs, with training and test accuracy climbing in tandem and converging above 90%. The close tracking between train and test accuracy throughout training (rather than train accuracy pulling for ahead of test accuracy) indicates the model is generalizing rather than overfitting.



2.2 Confusion Matrix

On the 30-sample test set, the model correctly classified all 10 Setosa and all 10 Virginica samples, and 9 of 10 Versicolor samples (only 1 Versicolor sample was misclassified as Virginica). This is consistent with well-known structure in the Iris dataset: Setosa is linearly separable from the other two classes, while Versicolor and Virginica have some feature overlap and are the pair most often confused by simple classifiers.



2.3 Findings and Insights

- Standardizing features (rather than leaving them in raw cm units) was important — it kept per-feature gradient magnitudes comparable and produced a noticeably smoother loss curve than an unscaled run.
- He initialization combined with ReLU gave stable, monotonically decreasing loss from epoch 1, with no dead-neuron collapse observed.
- Almost all of the remaining test-set error is concentrated in the Versicolor/Virginica boundary, matching the known geometry of the Iris feature space rather than indicating a bug in the implementation.
- Accuracy is a step function that plateaus for several epochs at a time (visible in the accuracy plot) even while loss keeps decreasing — a reminder that loss and accuracy don't always improve in lockstep, since loss reflects confidence while accuracy only reflects the arg-max decision.

3 Challenges Faced and Solutions Applied

- **Deriving the backward-pass gradients correctly:** the softmax + cross-entropy combination has a gradient that simplifies neatly to $(A_2 - Y)/m$, but it is easy to get this wrong if the two operations are differentiated separately. This was verified by checking that the training loss decreases smoothly and monotonically (a sign of a correct gradient) rather than diverging or oscillating.
- **Numerical stability in softmax and log:** computing softmax directly can overflow for large logits, and taking log(0) in cross-entropy is undefined. Both were handled — softmax by subtracting the row-wise maximum before exponentiating, and cross-entropy by clipping predicted probabilities away from exactly 0 or 1 before taking the log.

- **Meeting the accuracy target within the fixed hyperparameters** (100 epochs, batch size 16, learning rate 0.01): initial runs with some random seeds landed in the 80–90% range. Standardized preprocessing and He initialization were adopted (over raw features and naive random initialization) to make convergence more reliable within the fixed epoch budget, while keeping the required architecture and hyperparameters unchanged.

4 Answers to Questions

4.1 Why is ReLU used in the hidden layer instead of sigmoid or linear activation?

A purely linear activation would make the hidden layer redundant: stacking two linear transformations (input→hidden, hidden→output) is still just one linear transformation overall, so the network could only ever learn linear decision boundaries — insufficient for classes like Versicolor and Virginica, which overlap and need a non-linear boundary.

Sigmoid activation saturates for large positive or negative inputs, where its gradient approaches 0, and even at its steepest point the gradient is only 0.25. Backpropagating through several sigmoid units causes gradients to shrink multiplicatively — the vanishing gradient problem — slowing or stalling learning in earlier layers. ReLU's gradient is a constant 1 for all positive inputs, so gradient signal passes through unattenuated for active neurons, giving faster and more stable training. ReLU is also cheaper to compute (a comparison and max, vs. an exponential) and induces sparse activations, which can act as a mild regularizer.

4.2 If a model performs poorly on the test set, identify two possible causes and suggest improvements

Cause 1 — Overfitting: the model fits noise and idiosyncrasies of the training set rather than general patterns, so training accuracy is high while test accuracy lags well behind. *Improvements:* add L2 weight regularization, reduce the hidden layer's width, apply dropout, use early stopping based on a held-out validation set, or collect more training data.

Cause 2 — Poor hyperparameters or underfitting: a learning rate that is too high can cause unstable or diverging loss, while one that is too low (or too few epochs) can leave training incomplete within the epoch budget; an unsuitable batch size can also make gradient estimates noisy. *Improvements:* sweep the learning rate and compare loss curves, add a learning-rate schedule, monitor a validation set to judge whether more epochs would help, or increase model capacity (more hidden units, an additional hidden layer) if the model is too simple to separate the classes.

Other contributing factors worth checking: an unrepresentative train/test split (addressed here with stratified sampling), unscaled features distorting gradient magnitudes across features (addressed here with standardization) and poor weight initialization causing unstable early training or dead ReLU units (addressed here with He initialization).

5 AI Use Statement

Claude.ai was used for brainstorming and writing support. I used it to clarify technical concepts, mathematical concepts and formulas, verify understanding of concepts, and improve the clarity and structure of explanations.

For an example, I used claude to explain one-hot-encoding, mathematical explanation of ReLU. Furthermore i got help in interpreting the training loss, training accuracy, test accuracy.

In addition to that i got help with latex formatting, table generation and formulas.